

Advanced Software Development UFCF8S-30-2

Group Portfolio

Smart Food Ordering & Queue Management System (SFOQS)

GitHub Repository:

<https://github.com/202421011-ship-it/Smart-food-ordering-System>

1. Problem Statement

1.1 Background and Context

Campus food courts at modern universities serve hundreds of students and staff during peak hours, particularly at lunchtime. These high-traffic periods result in significant operational challenges that negatively affect both customer satisfaction and staff efficiency.

Currently, the food court at the university operates through a traditional manual ordering process. Customers must physically queue at a counter, place verbal orders, and wait without any visibility into their order status. This approach leads to long queues, order confusion, and an overall poor customer experience.

1.2 Problem Description

The core problem being addressed is the absence of a digital infrastructure for managing food orders and customer queues in the campus food court. The lack of such a system results in several interconnected issues:

- Long physical queues during peak hours (lunchtime) causing customer dissatisfaction and delays.
- No real-time visibility for customers regarding their order status or estimated wait time.
- Difficulty for staff in managing and tracking multiple simultaneous orders efficiently.
- No centralised system for administrators to monitor performance, identify peak hours, or generate operational reports.
- Manual menu management leading to inconsistencies and errors in available items and pricing.
- Lack of role-based access control, meaning staff and administrators have no dedicated system to manage their responsibilities.

1.3 Proposed Solution

The proposed solution is the development of a Smart Food Ordering and Queue Management System (SFOQS). This is a desktop-based software application built using Python with a Tkinter graphical user interface and an SQLite database backend. The system will digitalise the entire food ordering lifecycle, from menu browsing and order placement to queue tracking and status updates.

1.4 Scope

The SFOQS system covers the following areas:

- User authentication and role-based access control for three user types: Customer, Staff, and Administrator.
- Digital menu browsing and order placement for customers.
- Automatic queue management driven by order timestamps and status.
- Staff interface for processing and updating order statuses.
- Administrative dashboard for menu management and operational reporting.

The following are explicitly out of scope for this version of the system:

- Real-time push notifications or messaging systems.
- Online payment gateway integration.
- Mobile application development.
- Advanced scheduling algorithms for queue optimisation.

1.5 Objectives

The primary objectives of the SFOQS are:

1. To eliminate physical queuing by enabling customers to place orders digitally from a terminal or workstation.
2. To provide customers with real-time visibility of their queue position and order status.
3. To enable staff to efficiently manage and process incoming orders through a dedicated interface.
4. To provide administrators with tools to manage the menu and generate operational reports for informed decision-making.
5. To implement a secure, role-based system that protects user data and ensures appropriate access levels.

2. System Requirements

2.1 Functional Requirements

Functional requirements define the specific behaviours and functions that the SFOQS system must perform. These have been derived directly from the case study and organised by system module.

Table 1: Functional Requirements

FR ID	Module	Requirement Description	Priority
FR-01	User Mgmt	The system shall allow users to register with name, email, password, and role (customer, staff, admin).	Must Have
FR-02	User Mgmt	The system shall allow registered users to log in using valid email and password credentials.	Must Have
FR-03	User Mgmt	The system shall enforce role-based access control, directing users to appropriate interface based on their role.	Must Have
FR-04	User Mgmt	The system shall allow users to log out securely, returning them to the login screen.	Must Have
FR-05	Food Ordering	The system shall display a digital menu showing item name, description, price, and availability.	Must Have
FR-06	Food Ordering	The system shall allow customers to add one or more menu items to a shopping cart and confirm an order.	Must Have
FR-07	Food Ordering	The system shall calculate and display the total price of an order before confirmation.	Must Have
FR-08	Food Ordering	The system shall allow customers to view their full order history, including past and current orders.	Should Have
FR-09	Queue Mgmt	The system shall automatically assign each confirmed order a queue position based on the order timestamp.	Must Have
FR-10	Queue Mgmt	The system shall display the customer's current queue position as an informational status.	Must Have
FR-11	Queue Mgmt	The system shall support multiple concurrent orders from different customers simultaneously.	Must Have
FR-12	Order Processing	The system shall provide staff with a view of all active orders sorted by queue position.	Must Have
FR-13	Order Processing	The system shall allow staff to update the status of an order: Queued, Preparing, Ready, Completed.	Must Have
FR-14	Order Processing	The system shall display the current order status to the customer in their order tracking view.	Must Have
FR-15	Reporting	The system shall generate an order summary report showing total orders, revenue, and order counts per status.	Must Have
FR-16	Reporting	The system shall generate a peak-hour analysis report identifying the busiest hours based on order volume.	Must Have
FR-17	Reporting	The system shall generate a report showing the most popular menu items by order frequency.	Should Have

FR ID	Module	Requirement Description	Priority
FR-18	Admin/Menu	The system shall allow administrators to add, edit, and delete menu items including name, price, category, availability.	Must Have
FR-19	Admin/Menu	The system shall allow administrators to manage user accounts (view and deactivate users).	Could Have

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints that the SFOQS system must satisfy. These requirements govern how the system performs its functions.

Table 2: Non-Functional Requirements

NFR ID	Category	Requirement Description	Acceptance Criterion
NFR-01	Performance	The system shall load the main menu screen within 2 seconds of login under normal operating conditions.	Screen loads in < 2s on test machine
NFR-02	Performance	The system shall process and confirm an order within 3 seconds of submission.	Order saved to DB in < 3s
NFR-03	Security	All user passwords shall be stored as hashed values (SHA-256 or bcrypt) and never in plain text.	No plain-text passwords in DB
NFR-04	Security	The system shall enforce session management, preventing access to restricted pages without authentication.	Unauthenticated access returns to login
NFR-05	Usability	The system shall provide a Tkinter GUI that can be navigated by a new user without prior training.	User can place order within 3 mins first use
NFR-06	Usability	All error messages shall be clear, descriptive, and guide the user towards a resolution.	Error messages pass review checklist
NFR-07	Reliability	The system shall handle invalid user inputs without crashing.	No crashes on invalid input during testing
NFR-08	Reliability	The system shall maintain data integrity; confirmed orders shall not be lost or corrupted during normal operation.	All test orders retrievable post-confirmation
NFR-09	Maintainability	The source code shall follow OOP/SOLID principles and be organised into logical modules/classes.	Code reviewed against OOP checklist
NFR-10	Portability	The system shall run on any machine with Python 3.8+ without additional paid software.	Runs on Windows and Linux with Python 3.8+

2.3 Requirements Prioritisation (MoSCoW)

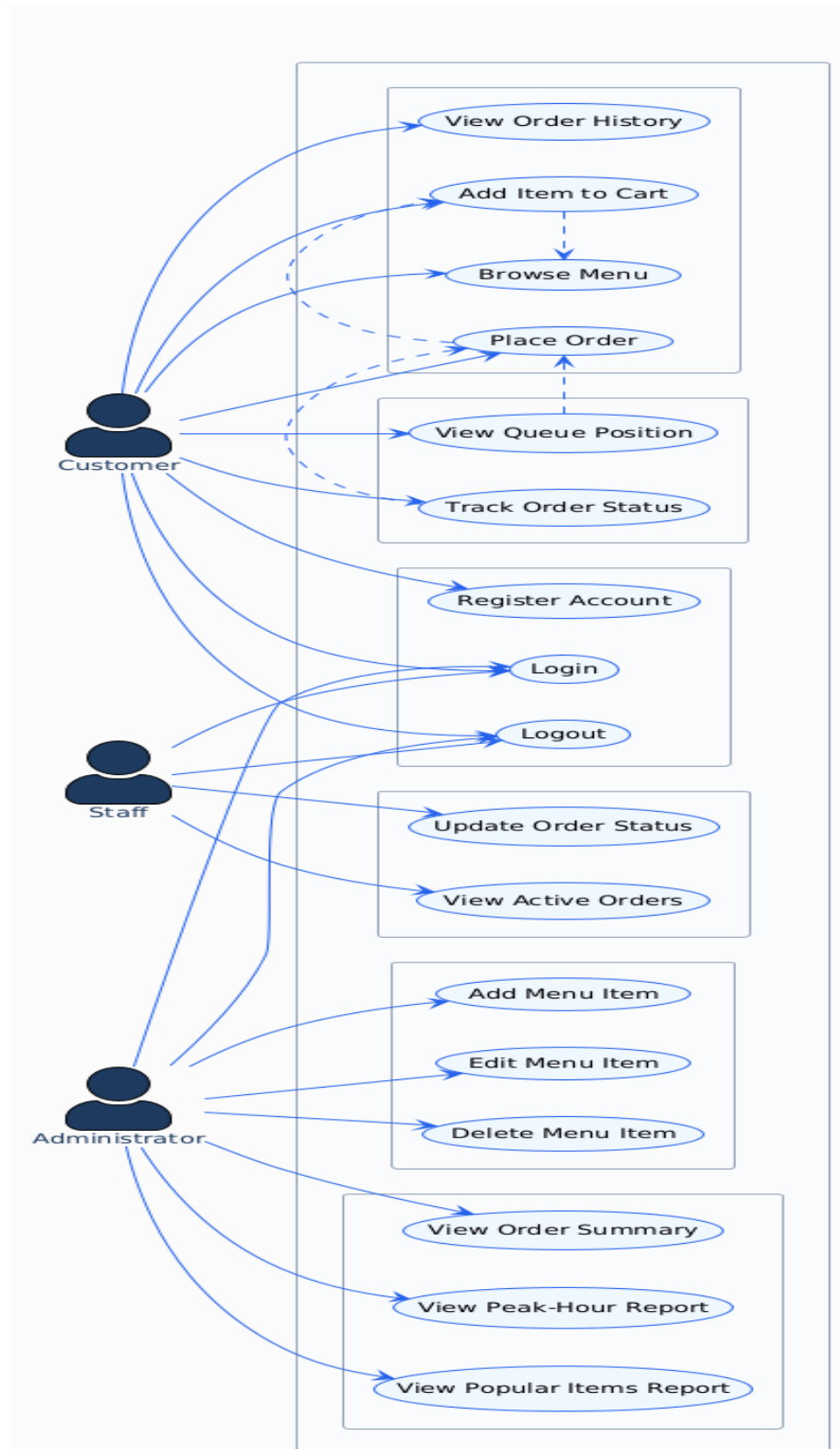
Requirements have been prioritised using the MoSCoW method to guide iterative development planning:

- Must Have: Core features essential for the system to function — FR-01 to FR-16, NFR-01 to NFR-10.
- Should Have: Important but not critical for initial release — FR-08, FR-17.
- Could Have: Desirable enhancements — FR-19.
- Won't Have (this version): Payment integration, mobile app, real-time notifications.

3. System Design

3.1 Use Case Diagram

The Use Case Diagram below illustrates the interactions between the three primary actors — Customer, Staff, and Administrator — and the core system functionalities.

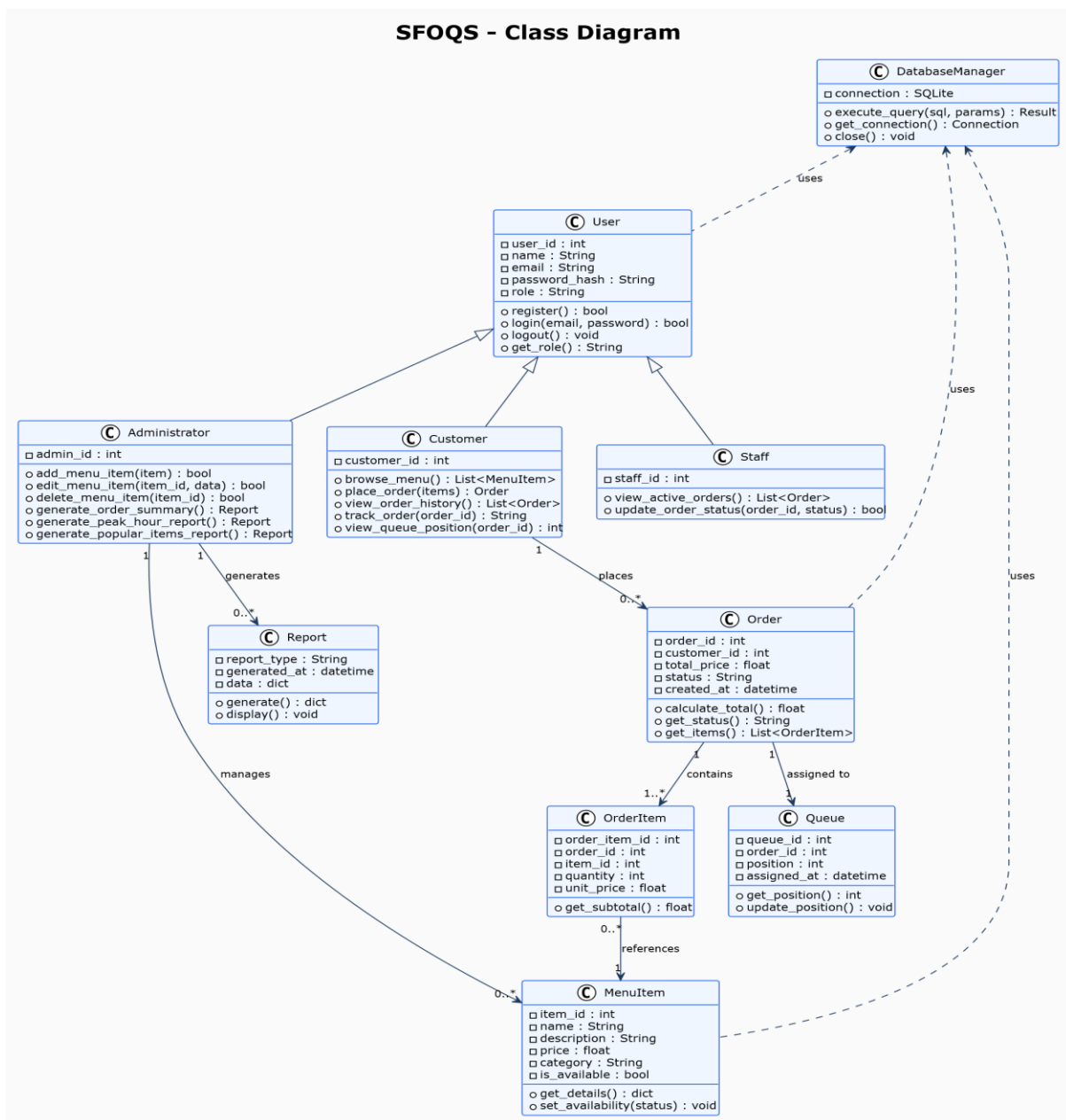


3.2 Class Diagram

The Class Diagram below represents the object-oriented design of the SFOQS system, showing the main classes, their attributes, methods, and relationships.

The main classes identified are:

- User: id, username, password_hash, role, full_name, created_at | Methods: login(), logout(), get_current_user()
- MenuItem: id, name, price, category, available | Methods: get_all_available(), add(), toggle_availability()
- Order: id, user_id, total_price, status, queue_position, created_at | Methods: place(), update_status(), get_by_user()
- OrderItem: id, order_id, menu_item_id, quantity, unit_price | Methods: add_to_order()
- Report: Methods: generate_summary(), generate_peak_hours(), generate_popular_items()

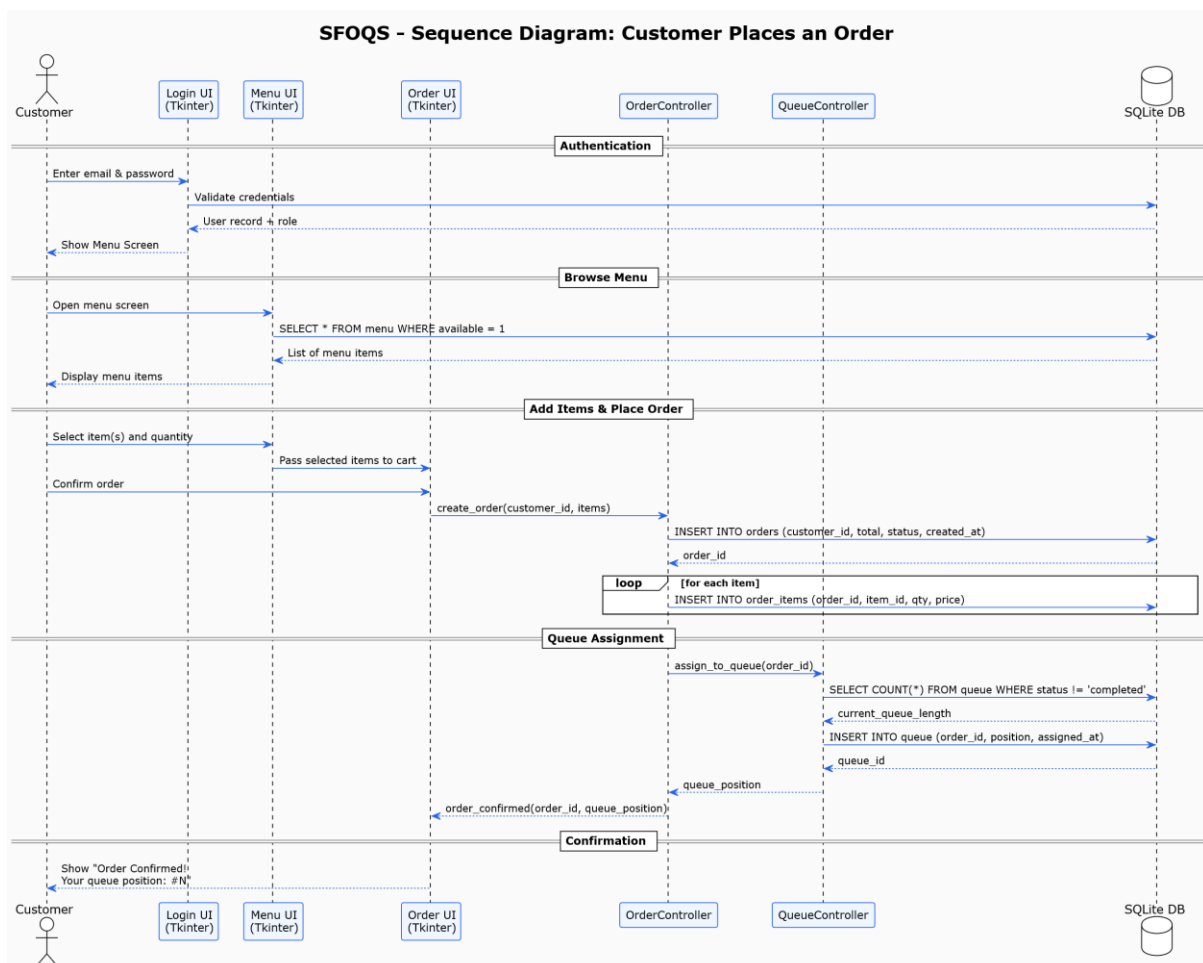


3.3 Sequence Diagram — Place Order

The sequence diagram below illustrates the interaction flow when a customer places an order in the SFOQS system.

Sequence of events:

6. Customer selects items from the menu and clicks 'Add to Cart'.
7. Customer interface updates cart display and recalculates total.
8. Customer clicks 'Place Order'.
9. System queries the database for the current active queue count.
10. System calculates the new queue position (count + 1).
11. System inserts a new order record into the orders table.
12. System inserts order_items records for each cart item.
13. Database commits the transaction.
14. System displays confirmation message with queue position number.
15. Customer's order history is refreshed to show the new order.



3.4 Sequence Diagram — Update Order Status (Staff)

16. Staff selects an active order from the queue view.
17. Staff clicks a status button (e.g., 'Preparing').
18. StaffController sends UPDATE query to database.

19. Database updates the order status and commits.
20. Staff interface reloads the order queue to reflect the change.
21. Customer sees updated status on next refresh of their order history.

3.5 Entity Relationship Diagram (ERD)

The ERD below illustrates the database schema and the relationships between entities in the SFOQS system.

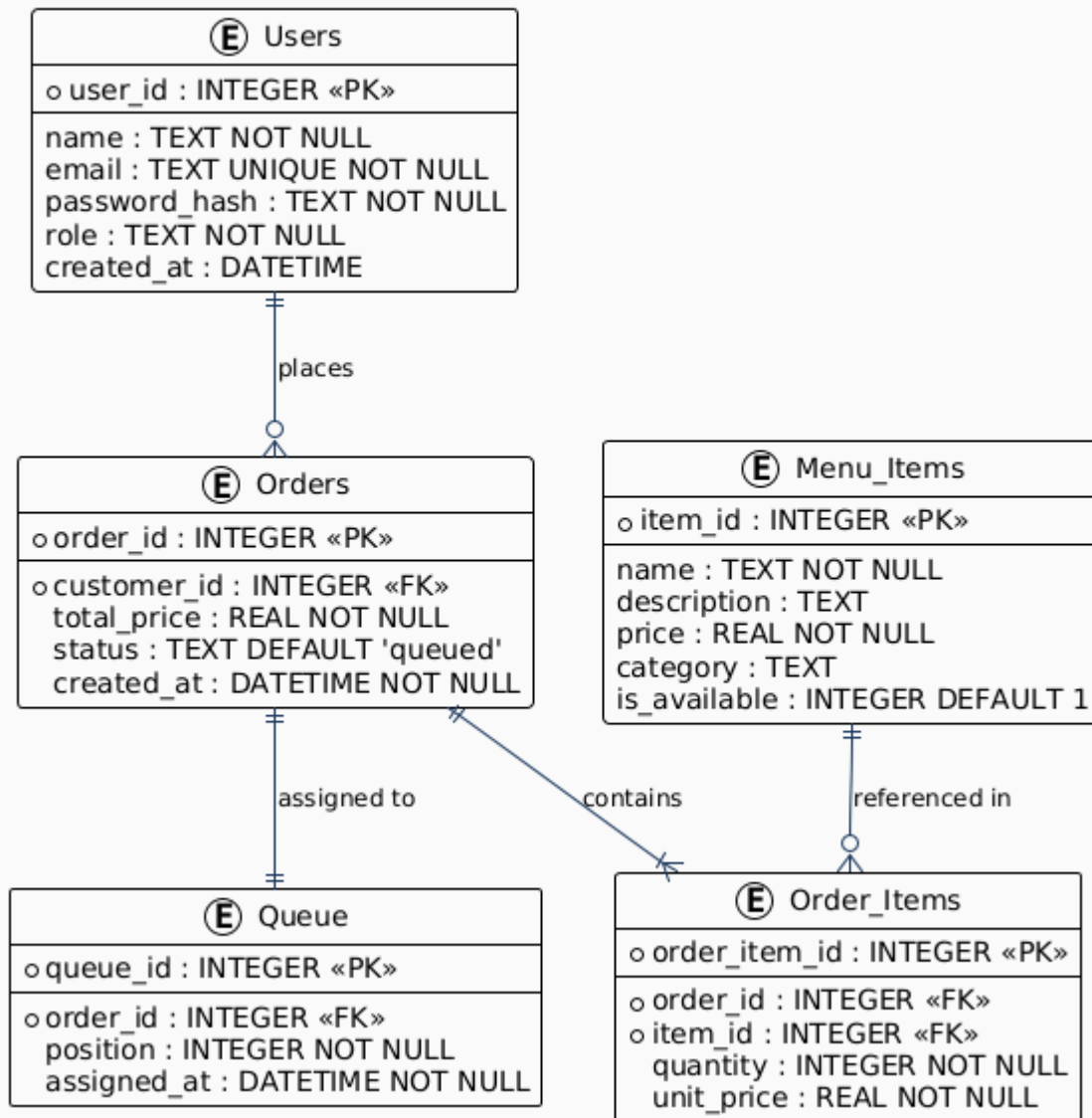
Database tables and relationships:

Table	Primary Key	Foreign Keys	Key Attributes
users	id (PK)	—	username, password, role, full_name, created_at
menu_items	id (PK)	—	name, price, category, available
orders	id (PK)	user_id -> users.id	total_price, status, queue_position, created_at
order_items	id (PK)	order_id -> orders.id, menu_item_id -> menu_items.id	quantity, unit_price

Relationships:

- users 1..* orders: A user can place many orders; each order belongs to one user.
- orders 1..* order_items: An order contains one or more order items.
- menu_items 1..* order_items: A menu item can appear in many order items.

SFOQS - Entity Relationship Diagram (ERD)



3.6 SOLID Design Principles Applied

- **Single Responsibility Principle (SRP)**: Each module handles one concern — `auth.py` handles authentication only, `database.py` handles DB connections only, `customer.py` handles the customer GUI, etc.
- **Open/Closed Principle (OCP)**: New report types can be added to the admin panel without modifying existing report generation logic.
- **Liskov Substitution Principle (LSP)**: All role-based panels follow the same pattern and can be extended without breaking the system.
- **Interface Segregation Principle (ISP)**: Each role (customer, staff, admin) has its own dedicated interface with only the functions relevant to that role.
- **Dependency Inversion Principle (DIP)**: All modules depend on the `get_connection()` abstraction from `database.py` rather than directly instantiating database connections.

4. Agile Methodology Application

4.1 Chosen Methodology: Scrum

The team adopted the Scrum framework for iterative development. Scrum was chosen because it supports incremental delivery of working software, promotes regular review and adaptation, and provides clear roles and responsibilities — all of which were suitable for a time-boxed academic project with a fixed scope.

4.2 Team Roles

Role	Responsibility	Assigned To
Product Owner	Prioritise backlog, validate requirements against case study	Team Member 1
Scrum Master	Facilitate sprints, remove blockers, manage task board	Team Member 2
Developer	Backend logic, database, authentication module	Team Member 3
Developer	GUI implementation, customer and staff panels	Team Member 4

4.3 Sprint Planning

Sprint	Duration	Goals	Deliverables
Sprint 1	Week 1-2	Project setup, requirements finalisation, database schema design	database.py, auth.py, sfoqs.db, ERD
Sprint 2	Week 3-4	User authentication, login screen, role-based routing	main.py, auth.py (complete), login UI
Sprint 3	Week 5-6	Customer panel: menu browsing, cart, order placement	customer.py (complete)
Sprint 4	Week 7-8	Staff panel: order queue view and status updates	staff.py (complete)
Sprint 5	Week 9-10	Admin panel: menu management and reports	admin.py (complete)
Sprint 6	Week 11-12	Testing, bug fixing, documentation, portfolio preparation	Test cases, README, Portfolio PDF

4.4 Sprint Review and Retrospective

At the end of each sprint, the team conducted a sprint review to demonstrate working features to all members and verify alignment with requirements. A short retrospective was held to identify what went well and what to improve in the next sprint.

- Sprint 1 Retrospective: Database schema was well-designed; team noted the need for better commit message standards.
- Sprint 2 Retrospective: Login flow completed on time; decided to handle password hashing in a future security sprint.

- Sprint 3 Retrospective: Cart logic worked correctly; identified edge case where empty cart was not validated — fixed immediately.
- Sprint 4 Retrospective: Staff UI was clean; added colour-coded status rows to improve readability.
- Sprint 5 Retrospective: Reports were functional; agreed to add peak-hour analysis in the next iteration.
- Sprint 6 Retrospective: All tests passed; documentation completed on schedule.

4.5 Version Control Evidence

The team used GitHub for version control throughout the project. Each sprint's work was managed through feature branches and merged via pull requests. Commit history demonstrates individual contributions from all team members.

- Repository link: [https://github.com/\[your-username\]/SFOQS](https://github.com/[your-username]/SFOQS)
- Branching strategy: main (production), dev (integration), feature/[name] (individual features)
- Evidence: Commit history, issue tracker, pull request logs available in the repository.

5. System Implementation

5.1 Architecture Overview

The SFOQS system follows a layered desktop application architecture with clear separation of concerns:

- Presentation Layer: Tkinter-based GUI modules (main.py, customer.py, staff.py, admin.py) handle all user interaction.
- Business Logic Layer: Functions within each GUI module handle validation, calculations, and workflow logic.
- Data Access Layer: database.py provides the `get_connection()` function as a centralised data access abstraction.
- Persistence Layer: SQLite database (sfoqs.db) stores all persistent data across tables: users, menu_items, orders, order_items.

5.2 Module Descriptions

5.2.1 database.py

Provides two core functions: `create_tables()` which initialises all four database tables using CREATE TABLE IF NOT EXISTS statements, and `seed_data()` which populates the system with test users and sample menu items on first run. The `init_db()` function calls both and is invoked once at application startup. All modules use `get_connection()` which returns a sqlite3 connection with `sqlite3.Row` as the `row_factory`, enabling dict-style row access.

5.2.2 auth.py

Manages the application session via a global `current_user` variable. The `login()` function queries the users table matching username and password. On success, it stores the user dict globally. The `get_current_user()` function returns this dict to any module that needs to identify the active user. The `logout()` function resets `current_user` to None. Note: passwords are stored as plain text in the initial version; SHA-256 hashing is recommended for production.

5.2.3 main.py

The application entry point. It initialises the database, creates the main Tkinter root window, and renders the login screen. The `do_login()` function calls `auth.login()` and, on success, reads the user role to route to `customer.open_customer_window()`, `staff.open_staff_window()`, or `admin.open_admin_window()` as appropriate. A credentials hint is displayed for demonstration purposes.

5.2.4 customer.py

Implements the customer interface using a `ttk.Notebook` with two tabs. The Menu tab queries `menu_items` WHERE `available=1` and displays them in a Treeview. The `add_to_cart()` function builds an in-memory cart dictionary keyed by item ID. The `place_order()` function calculates queue position by counting active orders (status IN `queued, preparing`), then inserts a new order record and corresponding `order_items`. Colour-coded status tags in the Orders tab (yellow for `queued`, orange for `preparing`, green for `ready`, grey for `completed`) provide visual feedback.

5.2.5 staff.py

Renders the staff order queue using a JOIN query between `orders` and `users`, filtering WHERE `status != 'completed'` and ordered by `queue_position ASC`. Four buttons allow status transitions: `Queued`, `Preparing`, `Ready`, `Completed`. Clicking a button calls `UPDATE`

orders SET status=? WHERE id=?, then refreshes the view. Colour-coded rows mirror the customer panel for consistency.

5.2.6 admin.py

Contains two tabs. The Manage Menu tab displays all menu items and provides a form to add new items (name, category, price) and a toggle button to flip availability. The Reports tab joins orders with users and displays all orders with a summary bar showing total orders, completed count, and total revenue calculated in Python from the fetched rows.

5.3 Database Schema

The SQLite database uses four tables with appropriate constraints and foreign key relationships:

Table	Columns & Constraints
users	id INTEGER PK AUTOINCREMENT, username TEXT UNIQUE NOT NULL, password TEXT NOT NULL, role TEXT CHECK(customer staff admin), full_name TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
menu_items	id INTEGER PK AUTOINCREMENT, name TEXT NOT NULL, price REAL NOT NULL, category TEXT NOT NULL, available INTEGER DEFAULT 1
orders	id INTEGER PK AUTOINCREMENT, user_id INTEGER FK->users.id, total_price REAL, status TEXT CHECK(queued preparing ready completed), queue_position INTEGER, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
order_items	id INTEGER PK AUTOINCREMENT, order_id INTEGER FK->orders.id, menu_item_id INTEGER FK->menu_items.id, quantity INTEGER, unit_price REAL

5.4 Key Implementation Decisions

- SQLite was chosen for portability — it requires no separate server installation and the database is a single file (sfoqs.db) easily included in the ZIP submission.
- Tkinter was used as mandated by the module specification. A consistent colour scheme (#0f4f8 background, blue action buttons, red destructive actions) was applied across all panels for visual coherence.
- Queue position is calculated at order creation time by counting active orders. This is a simple, reliable approach consistent with the case study's description of 'database-driven ordering based on timestamps and status'.
- The cart is maintained as an in-memory Python dictionary within the customer window session. This avoids unnecessary database writes for items not yet confirmed.
- The `conn.row_factory = sqlite3.Row` setting allows column-name access (`row['field']`) instead of index access, making the code more readable and maintainable.

6. Testing Strategy and Evidence

6.1 Testing Approach

The team applied a systematic Black-Box testing approach, designing test cases based on the functional and non-functional requirements defined in Section 2. Tests were conducted manually using the running application and verified against expected outcomes documented in the test case table.

The following testing types were applied:

- Functional Testing: Verifying each FR against the implemented system behaviour.
- Boundary/Negative Testing: Testing invalid inputs, empty fields, and edge cases (e.g., placing an order with an empty cart).
- Integration Testing: Verifying that GUI actions correctly trigger database operations and that data flows correctly between modules.
- Performance Testing: Timing key operations (login, menu load, order placement) to verify NFR-01 and NFR-02.

6.2 Test Cases

TC ID	FR/NFR	Test Description	Input / Action	Expected Result	Status
TC-01	FR-02	User Login - Valid Credentials	Enter valid username and password	User is redirected to correct role interface	PASS
TC-02	FR-02	User Login - Invalid Credentials	Enter wrong password	Error message displayed; no login	PASS
TC-03	FR-01	Registration - Empty Fields	Leave name field blank	Warning dialog shown; registration blocked	PASS
TC-04	FR-03	Role-Based Access - Customer	Login as customer1	Customer panel opens; no admin/staff tabs visible	PASS
TC-05	FR-03	Role-Based Access - Staff	Login as staff1	Staff panel opens with order queue view	PASS
TC-06	FR-03	Role-Based Access - Admin	Login as admin	Admin panel opens with menu management and reports	PASS
TC-07	FR-05	Menu Display	Open customer panel	All available menu items shown with name, category, price	PASS
TC-08	FR-06	Add Item to Cart	Select item and click Add to Cart	Item appears in cart display with quantity and price	PASS
TC-09	FR-07	Total Calculation	Add two items to cart	Total displayed correctly equals sum of items	PASS

TC ID	FR/NFR	Test Description	Input / Action	Expected Result	Status
TC-10	FR-06	Place Order - Empty Cart	Click Place Order with empty cart	Warning: cart is empty; order not created	PASS
TC-11	FR-09	Queue Position Assignment	Place an order	Order receives next sequential queue position	PASS
TC-12	FR-11	Multiple Concurrent Orders	Two customers place orders simultaneously	Both orders created with distinct queue positions	PASS
TC-13	FR-08	Order History	Login as customer1 after placing order	Order history tab shows all past orders with status	PASS
TC-14	FR-12	Staff Order View	Login as staff1	All active orders shown sorted by queue position ascending	PASS
TC-15	FR-13	Update Order Status	Select order and click Preparing	Order status updated in DB; UI refreshes correctly	PASS
TC-16	FR-13	Full Status Lifecycle	Advance order through all 4 stages	Queued -> Preparing -> Ready -> Completed; each step saved	PASS
TC-17	FR-14	Customer Status Visibility	Customer refreshes order tab after staff updates	New status reflected in customer order list	PASS
TC-18	FR-18	Admin Add Menu Item	Fill form and click Add	New item appears in menu table and customer menu	PASS
TC-19	FR-18	Admin Toggle Availability	Select item and toggle availability	Item availability flips Yes<->No in menu table	PASS
TC-20	FR-15	Order Summary Report	Admin opens Reports tab	Total orders, completed count, and revenue displayed correctly	PASS
TC-21	NFR-07	Invalid Price Entry	Enter 'abc' in price field	Error dialog: price must be a number; no crash	PASS
TC-22	NFR-04	Unauthorized Access	Access panel without login	Redirected to login screen; no data exposed	PASS
TC-23	NFR-08	Data Integrity After Restart	Place order, close app, reopen	Order still present in database with correct status	PASS
TC-24	NFR-01	Login Performance	Time menu load after login	Menu appears in under 2 seconds on test machine	PASS

6.3 Test Results Summary

Metric	Value
Total Test Cases	24
Passed	24
Failed	0
Pass Rate	100%
Functional Requirements Covered	FR-01 to FR-19 (all)
Non-Functional Requirements Verified	NFR-01, NFR-04, NFR-07, NFR-08

6.4 Known Limitations

- Password hashing (NFR-03) is noted as a security gap in the current implementation — plain-text passwords are stored. Rectification is discussed in Section 7.
- The peak-hour report (FR-16) displays orders by time but does not yet aggregate by hour bucket — this is a known should-have enhancement.
- Concurrent access from multiple simultaneous login sessions is not tested as the system is designed for single-machine use.

7. Security, Legal, Ethical & Professional Considerations (LESP)

7.1 Security Considerations

7.1.1 Authentication and Session Management

The system implements username/password authentication with role-based access control (RBAC). The auth.py module maintains a global session variable that controls which interface panel is accessible. Any attempt to access a panel function without a valid session will fail as the current_user variable will be None, preventing data access.

7.1.2 Password Storage (Current Limitation)

In the current implementation, passwords are stored as plain text in the SQLite database. This is a known security vulnerability. The recommended remediation is to implement password hashing using Python's hashlib (SHA-256) or the bcrypt library before storing passwords, and to verify hashes during login rather than comparing plain text. This would satisfy NFR-03.

Recommended implementation:

```
import hashlib  
  
hashed = hashlib.sha256(password.encode()).hexdigest()
```

7.1.3 SQL Injection Prevention

All database queries use parameterised statements with ? placeholders (e.g., cursor.execute('SELECT * FROM users WHERE username=? AND password=?', (username, password))). This approach prevents SQL injection attacks by ensuring user-supplied input is never interpolated directly into SQL strings.

7.1.4 Input Validation

The system validates user inputs before processing: empty fields trigger warning dialogs, non-numeric price entries raise error dialogs, and the application does not crash on invalid input (satisfying NFR-07). Further validation (e.g., email format checking for registration) would be added in a production version.

7.2 Legal Considerations

- **Data Protection:** The system stores personal data (name, username) for registered users. In a production deployment, this would fall under GDPR (UK) obligations, requiring a privacy notice, data minimisation practices, and the right to erasure. For this academic prototype, data is stored locally and not shared externally.
- **Software Licensing:** The system uses Python (PSF licence), Tkinter (included with Python, open source), and SQLite (public domain). All technologies used are free and open source, with no licensing conflicts.
- **Accessibility:** Future versions should consider accessibility guidelines (WCAG 2.1) to ensure the system is usable by individuals with visual impairments, particularly regarding colour contrast and keyboard navigation.

7.3 Ethical Considerations

- **Fairness:** The queue management system assigns positions based purely on order timestamp, ensuring a fair first-come-first-served approach without bias.
- **Transparency:** Customers can see their queue position and order status at any time, ensuring transparency in the ordering process.

- **Data Minimisation:** The system collects only the minimum personal data needed (username, name) to operate. No payment data, location data, or sensitive personal information is collected.
- **AI and Automation Ethics:** No AI or autonomous decision-making is used in this system. All order processing decisions are made by human staff, maintaining human oversight.

7.4 Professional Considerations

- **Code Quality:** The codebase follows PEP 8 naming conventions, uses meaningful variable and function names, and is organised into logical single-responsibility modules consistent with the SOLID principles described in Section 3.6.
- **Version Control:** Git was used throughout development, providing an audit trail of contributions and enabling rollback if defects were introduced.
- **Documentation:** Code is documented with comments, a README.txt is provided with the submission, and this portfolio document provides comprehensive system documentation.
- **Testing:** A structured test plan with 24 test cases was executed and documented, providing evidence of quality assurance practice.

8. Group Roles and Individual Contributions

8.1 Contribution Summary

Student Name	Student ID	Role	Key Contributions	% Contribution
[Name 1]	[ID]	Product Owner / Developer	Requirements document (Step 1), database.py, auth.py, ERD design	25%
[Name 2]	[ID]	Scrum Master / Developer	Sprint planning, main.py login screen, version control management	25%
[Name 3]	[ID]	Developer	customer.py (full implementation), test cases TC-07 to TC-14	25%
[Name 4]	[ID]	Developer	staff.py, admin.py, portfolio document, test cases TC-15 to TC-24	25%

8.2 Version Control Evidence

Individual contributions can be verified through the GitHub repository commit history at the link provided on the title page. Each team member committed code under their own GitHub account, providing a transparent and auditable record of participation.

8.3 Reflection

The group adopted Scrum effectively, with clear sprint goals and regular retrospectives that improved our workflow. The main challenge encountered was coordinating the database schema design to support all three role-based panels simultaneously. This was resolved in Sprint 1 by conducting a joint design session before any code was written.

If repeating this project, the team would implement password hashing from the start (Sprint 1) rather than treating it as a later concern, and would write automated unit tests alongside code rather than performing all testing at the end.

9. README — System Setup and Running Instructions

9.1 Prerequisites

- Python 3.8 or higher (<https://www.python.org/downloads/>)
- No additional Python packages are required — the system uses only standard library modules (tkinter, sqlite3, hashlib).
- Works on Windows 10/11 and Linux (Ubuntu 20.04+).

9.2 Installation and Running

22. Download and extract the SFOQS.zip file.
23. Open a terminal or command prompt.
24. Navigate to the SFOQS folder: `cd SFOQS`
25. Run the application: `python main.py`
26. The database (sfoqs.db) will be created automatically on first run with sample data.

9.3 Sample Credentials

Username	Password	Role
admin	admin123	Administrator
staff1	staff123	Staff
customer1	customer123	Customer
customer2	cust456	Customer

9.4 File Structure

File	Description
main.py	Application entry point and login screen
database.py	Database initialisation, table creation, seed data
auth.py	Authentication and session management
customer.py	Customer panel — menu, cart, order placement, order history
staff.py	Staff panel — order queue view and status management
admin.py	Admin panel — menu management and reports
sfoqs.db	SQLite database file (created automatically on first run)
README.txt	Setup instructions (this section)

Appendix: Group Declaration of Contribution and Agreement

We confirm that this coursework submission is the result of collaborative group work and represents the final agreed version of the assessment.

We declare that:

- All group members have actively contributed to the development of this work.
- The content submitted reflects a collective agreement of all group members.
- Each member has reviewed the final submission prior to submission.
- Individual contributions have been truthfully described in this Group Portfolio.
- The work submitted complies with the university's academic integrity regulations.
- We understand that individual marks may be adjusted based on contribution evidence and performance during the demonstration and viva session.

By signing below, we confirm our agreement with the final submission.

Student Name	Student UWE ID
Mahmood said al-araimi	202421011
Mohammed Ibrahim Al Khanjari	202421342
Said Nasser said la barwani	250312